Analysis of IoT Botnet Structure and Behavior: A Study Based on the Hitori (SlumX) Variant

Ariel Matias Melo Pazmiño
Universidad Internacional del Ecuador - Facultad de Ciencias Técnicas

Abstract

This research presents a comprehensive technical analysis of the SlumX botnet (Hitori variant), elaborating on its architecture, attack vectors, and operational patterns through static code analysis. The study unmasks a sophisticated Command and Control (C2) mechanism, diverse DDoS attack methods (UDP, TCP, HTTP), and advanced evasion techniques unique to IoT devices [1]. Key findings include the identification of multi-architecture support (MIPS, ARM, x86, etc.) and the evolution of Mirai-driven botnets toward more complex infrastructures with increased persistence [2]. This work contributes to cybersecurity research by documenting modern IoT botnet behaviors and suggesting mitigation strategies [5].

Key Focus: Understanding the evolution from simple credential attacks to sophisticated multi-architecture IoT malware with advanced evasion capabilities.

Research Objectives

This study aims to dissect the operational mechanics of the SlumX botnet, a variant of the infamous Mirai malware [1]. By analyzing the source code, we identify the specific modifications that allow it to target a wider range of IoT devices compared to its predecessors.

Specific Objectives

- Analyze core architecture and operational framework.
- Examine Command & Control infrastructure design.
- Identify and categorize DDoS attack vectors [3].
- Evaluate multi-architecture support mechanisms.
- Document evasion and anti-analysis techniques.

Methodology

Static Code Analysis Framework

The research employs a static analysis approach, examining the C and Python source files from the `mat1520/slumpx` repository. This method allows for the identification of hardcoded values, command structures, and logic flows without the risks associated with executing live malware [2].

Analysis of the `mat1520/slumpx` repository:

- **Code Review:** Systematic examination of source files (`bot.c`, `server.c`, `scankill.c`).
- **Control Flow Analysis:** Mapping execution paths and decision points.
- **Threat Modeling:** Assessment of attack capabilities and defensive gaps.

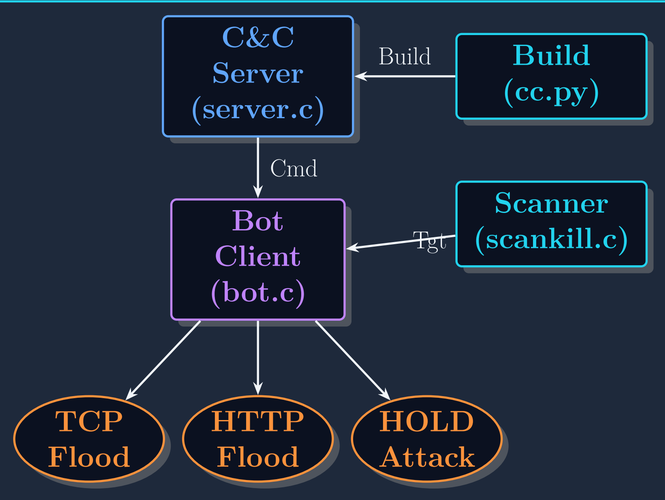
Threat Model

Threat Modeling Framework

- **Attack Surface:** 12+ architectures, 7+ attack vectors.
- **Impact:** Potential for >1Tbps DDoS traffic generation.
- **Vulnerabilities:** Buffer overflows in `sprintf()`, lack of thread synchronization.

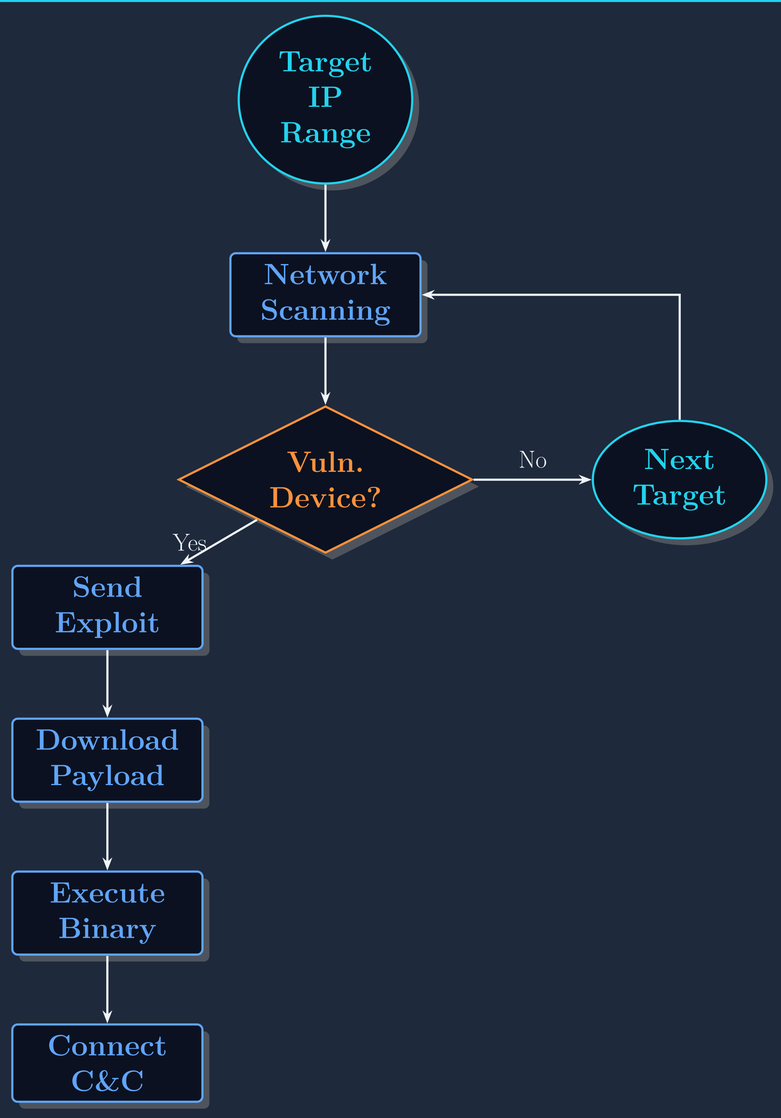
Limitations: Static analysis cannot capture all runtime behaviors; no testing on actual IoT devices.

System Architecture



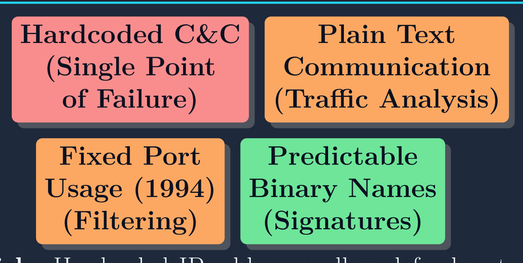
The architecture follows a centralized C2 model [1]. The `server.c` component manages the botnet via TCP port 1994, while `bot.c` resides on infected devices. The `cc.py` build system is crucial for cross-compiling payloads for various architectures.

Infection Process Flow



The infection cycle begins with network scanning (`scankill.c`) to identify vulnerable devices. Upon successful exploitation (e.g., via Telnet brute-force or specific CVEs), the payload is downloaded and executed, establishing a persistent connection to the C2 server [2].

Security Risk Assessment



High Risk: Hardcoded IP addresses allow defenders to sinkhole the entire botnet. **Medium Risk:** Lack of encryption enables traffic inspection and signature generation.

Multi-Architecture Support

MIPS: Routers, Network Equip. **ARM:** IoT Devices, IP Cameras

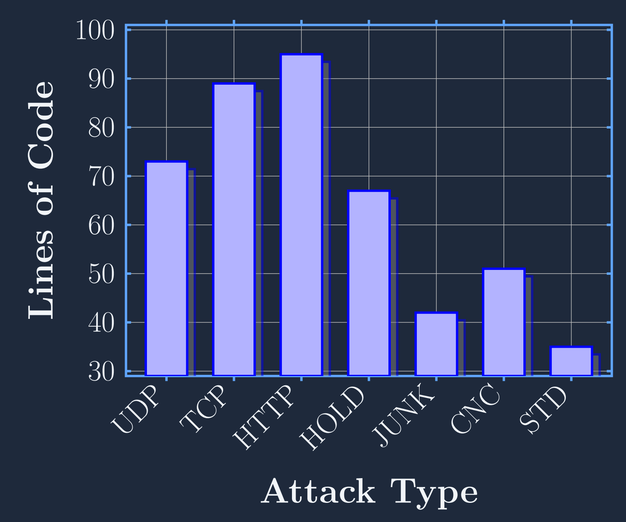
x86/x64: PC-based IoT Systems **PowerPC:** Legacy Embedded

SPARC: Enterprise IoT Devices **SH4:** Multimedia Devices

12 architectures supported - Massive IoT attack surface

Unlike early IoT botnets that targeted specific platforms, SlumX supports 12 distinct architectures including MIPS, ARM, and x86 [3]. This broad compatibility significantly increases the potential infection pool, as noted in recent IoT statistics [5].

Attack Vector Complexity



HTTP-based attacks show highest complexity (95 LOC).

The botnet supports multiple DDoS vectors. Analysis shows that HTTP floods are the most complex to implement (95 LOC), requiring valid header construction to bypass simple filters. In contrast, UDP and TCP floods rely on raw socket manipulation for high-volume traffic generation [3].

Key Findings & Conclusions

- **Multi-architecture:** 12 architectures (MIPS, ARM, x86, embedded) dramatically expanding attack surface.
- **Advanced evasion:** Process masquerading, anti-analysis features, IoT-specific hiding techniques.
- **Targeted exploitation:** Vendor-specific exploits showing evolution to targeted attacks.
- **Future Trends:** Mirai-driven botnets are moving toward more sophisticated infrastructures with increased persistence [4].

Future Research:

- Dynamic analysis frameworks
- Specialized detection technologies [4]

References

1 M. Antonakakis et al., "Understanding the Mirai Botnet," *USENIX Security*, 2017.

2 A. Marzano et al., "The evolution of bashlite and mirai IoT botnets," *IEEE S&P*, 2018.

3 G. Vishwakarma and G. S. Chhabra, "Targeted DDoS Attacks on IoT: A Survey of the Mirai Botnet and its Variants," *AISC*, 2023.

4 A. D. Alghazzawi et al., "An Efficient Hybrid Deep Learning Model for Detecting IoT Botnet Attacks," *IEEE Access*, 2024.

5 Statista, "IoT connected devices installed base worldwide," 2024.

Research Paper Access